

Docker Networks

List Networks

```
docker networks
```

Common defaults:

- `bridge` (default)
- `host`
- `none`

Inspect Network

```
docker network inspect <network_name>
```

Shows:

- Subnet / Gateway
- Connected containers
- Driver type
- Internal DNS info

👉 Use this when containers **can't talk to each other**.

Bridge Network (Most Important)

What is `bridge` ?

- Default Docker network
- Containers get **private IPs**
- Containers on same bridge can talk via **container name (DNS)**

User-defined bridge

```
docker network create -d bridge my_bridge
```

Run container on it:

```
docker run -it --network my_bridge ubuntu
```

- ✓ Automatic DNS
 - ✓ Container name → hostname
 - ✓ Isolation
-



Docker Volumes

Why Volumes?

- Containers are **ephemeral**
 - Volumes persist data
 - Used for DBs, logs, uploads
-

Volume Commands

List Volumes

```
docker volume ls
```

Create Volume

```
docker volume create myvolume
```

Inspect Volume

```
docker volume inspect myvolume
```

Remove Volume

```
docker volume rm myvolume
```

Remove All Unused Volumes

```
docker volume prune
```

Using Volumes

Named Volume

```
docker run -v myvolume:/data ubuntu
```

Bind Mount (host → container)

```
docker run -v $(pwd):/app ubuntu
```

Type	Use Case
Volume	Production, DBs
Bind Mount	Local dev

Volumes in Docker Compose

```
services:
  postgres:
    image: postgres
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
volumes:  
pgdata:
```

Multi-Stage Docker Build

Why Multi-Stage?

- Smaller images
- No build tools in final image
- Faster, cleaner, safer

Problem (Single Stage)

```
FROM node:18  
RUN npm install  
RUN npm run build  
CMD ["node", "server.js"]
```

✗ Includes node_modules, build tools, junk

✗ Large image

Solution (Multi-Stage)

Syntax

```
# Build stage  
FROM node:18 AS builder  
WORKDIR /app  
COPY package*.json .  
RUN npm install
```

```
COPY . .  
RUN npm run build  
  
# Runtime stage  
FROM node:18-slim  
WORKDIR /app  
COPY --from=builder /app/dist ./dist  
CMD ["node", "dist/index.js"]
```

Key Concepts

- `AS builder` → stage name
- `COPY --from=builder` → copy only needed artifacts
- Final image has **zero build dependencies**

✓ Smaller image

✓ Faster pull

✓ Production-grade

Build

```
docker build -t myapp .
```

Docker **automatically discards** intermediate stages.